

LaboEscape — comentários sobre os algoritmos

Entrega 2

1. O que o programa faz no geral

O jogo é um quiz de terminal. O jogador acorda em um laboratório e precisa responder 3 perguntas para escapar. Cada erro tira 25 de energia. Se a energia zerar, o jogador perde.

O fluxo é basicamente este:

Main() começa e mostra a introdução.

Fase 1 -> pergunta sobre Tutankâmon.

Fase 2 -> pergunta sobre a Antártida.

Fase 3 -> pergunta sobre o dólar.

Mostra o final dependendo de como foi a partida.

2. A função Digitar() — efeito de digitação

Essa função pega um texto e imprime letra por letra, com uma pausa de 30ms entre cada uma. É isso que dá aquele visual de terminal digitando.

```
static void Digitar(string texto)
{
    foreach (char c in texto)
    {
        Console.Write(c); // imprime uma letra
        Thread.Sleep(30); // espera 30 milissegundos
    }
    Console.WriteLine(); // pula linha no final
}
```

Obs.: o foreach percorre cada caractere do texto um por um. O Thread.Sleep(30) é o que cria o efeito — sem ele, tudo apareceria de uma vez.

3. As variáveis que controlam o jogo

Três variáveis guardam o estado do jogo do começo ao fim:

```
int energia = 100; // vida do jogador, começa em 100
int pontos = 0; // pontuação, sobe 10 por acerto
bool jogando = true; // true = jogo continua, false = acabou
```

Obs.: int é usado para números inteiros e bool para verdadeiro ou falso. Essas três variáveis são lidas e alteradas durante as três fases.

4. Os loops while — como cada fase funciona

Cada fase usa um while para repetir a pergunta até o jogador acertar, errar demais ou querer sair. O while só continua rodando enquanto a condição for verdadeira.

```
while (jogando) // roda enquanto jogando for true
{
    // mostra a pergunta e as opções
    string r = Console.ReadLine(); // lê o que o jogador digitou

    if (r == "0") // quer sair
    {
        jogando = false;
        break; // sai do loop na hora
    }
    else if (r == "1") // acertou (exemplo da fase 1)
    {
        pontos += 10;
        break;
    }
    else // errou
    {
        energia -= 25;
    }

    if (energia <= 0) // zerou a energia?
    {
        jogando = false;
        break;
    }
}
```

Obs.: o break força a saída do loop imediatamente. Sem ele, o jogo ficaria preso repetindo a mesma pergunta para sempre.

5. if / else if / else — tomando decisões

O programa usa if/else if/else para decidir o que fazer com cada resposta. A lógica é: se for X, faz isso; se não for X, mas for Y, faz aquilo; se não for nenhum dos dois, faz outra coisa.

```
if (r == "0") // se digitou 0
    jogando = false; // marca para sair

else if (r == "2") // se digitou 2 (certo na fase 2)
    pontos += 10; // ganha pontos

else // qualquer outra coisa
    energia -= 25; // perde energia
```

Obs.: as condições são testadas em ordem. Assim que uma for verdadeira, ele executa aquele bloco e ignora os outros.

6. Como as fases se encadeiam

A fase 2 só começa se a fase 1 terminar sem o jogo acabar. A mesma coisa acontece com a fase 3. Isso é feito com um if antes de cada fase.

```
// fase 1 acontece sempre
while (jogando) { /* fase 1 */ }

// fase 2 só roda se o jogo não acabou
if (jogando && energia > 0)
{
    while (jogando) { /* fase 2 */ }
}

// fase 3 idem
if (jogando && energia > 0)
{
    while (jogando) { /* fase 3 */ }
}
```

Obs.: o && é o operador lógico “E”: as duas condições precisam ser verdadeiras ao mesmo tempo. Se uma delas for falsa, a fase não executa.

7. Os três finais possíveis

No final, o programa decide qual ending mostrar com base em como o jogo terminou.

```
if (energia <= 0) // perdeu toda a energia -> final ruim
else if (!jogando) // saiu voluntariamente -> final intermediário
else // acertou tudo -> final bom
```

Obs.: o ! antes de jogando representa negação. !jogando significa “jogando é false”. Isso identifica o caso de quem escolheu sair sem ter zerado a energia.

8. Thread.Sleep() — as pausas de suspense

Além do Digitar(), o Thread.Sleep() aparece sozinho várias vezes no código para criar ritmo na narrativa. Pausas mais longas aparecem nos momentos mais tensos.

```
Digitar(">> as luzes apagam.");
Thread.Sleep(600); // 0,6 segundo de silêncio
Digitar(">> não todas.");
Thread.Sleep(900); // um pouco mais de tensão
Digitar(">> tem algo segurando seus pés.");
```

Obs.: 1000ms = 1 segundo. É uma escolha de design: o código não é só lógica, ele também conta a história pelo ritmo.

9. Console — entrada e saída

O jogo usa quatro comandos para se comunicar com o jogador:

```
Console.Clear(); // limpa a tela (só no início)
Console.Write(c); // imprime sem pular linha (usado no Digitar)
Console.WriteLine(); // pula linha
Console.ReadLine(); // lê o que o jogador digitou
```

Obs.: Console.ReadLine() devolve uma string, por isso a variável r é string e a comparação usa aspas: r == "1".

10. Resumo rápido

Cada parte do jogo usa um conceito diferente:

- Digitar() -> foreach + Thread.Sleep
- energia / pontos -> variáveis int
- jogando -> variável bool
- cada fase -> while + break
- respostas -> if / else if / else
- encadeamento -> if (jogando && energia > 0)
- finais -> if / else if / else
- pausas -> Thread.Sleep()
- entrada/saída -> Console.Read / Write / WriteLine

São os blocos básicos de qualquer programa: variáveis, loops, decisões e entrada/saída. O LaboEscape usa tudo isso junto de um jeito simples.

Fim.